

1.4 Numpy与多项式运算

1.4.1 多项式的构造与运算

`numpy.poly1d` 是NumPy中用于表示一维多项式的类。你可以通过系数来创建多项式，并使用这个类进行多项式的计算（如加法、乘法、求值等）。

```
import numpy as np

# 创建一个多项式  $2x^2 + 3x + 1$ 
p = np.poly1d([2, 3, 1])

# 显示多项式
print(p)

# 计算多项式在x=5的值
print(p(5))

# 多项式的加法
q = np.poly1d([1, 2]) #  $x + 2$ 
print(p + q)

# 多项式的乘法
print(p * q)
```

使用 `numpy.poly1d` 创建的多项式对象可以直接进行四则运算（加、减、乘、除），这些操作会返回新的 `numpy.poly1d` 对象，表示结果多项式。

下面是如何使用 `numpy.poly1d` 进行多项式四则运算的示例：

```

import numpy as np

# 创建两个多项式
p = np.poly1d([2, 3, 1]) # 表示  $2x^2 + 3x + 1$ 
q = np.poly1d([1, 2])    # 表示  $x + 2$ 

# 加法
p_plus_q = p + q
print("p + q =", p_plus_q)

# 减法
p_minus_q = p - q
print("p - q =", p_minus_q)

# 乘法
p_times_q = p * q
print("p * q =", p_times_q)

# 注意：除法在多项式运算中比较特殊，因为除法的结果可能包括商和余数
# numpy.poly1d 提供了 divmod 函数，用于同时返回商和余数
q_div_p, remainder = np.poly1d.divmod(q, p)
print("q / p (商) =", q_div_p)
print("余数 =", remainder)

# 如果你只想要商（不考虑余数），可以使用 div 函数（但在 numpy.poly1d 中没有直接的 div 方法，
# 你需要手动计算或使用 divmod 并只取第一个返回值）
# 注意：这里我们使用 divmod 并只取商
q_div_p_only_quotient = np.poly1d.divmod(q, p)[0]
print("q / p (仅商) =", q_div_p_only_quotient)

# 但是，通常对于多项式除法，我们更关心的是商，因此可以像上面那样处理

```

多项式除法的结果可能不是整数或简单的有理数，而是一个新的多项式（商）和一个余数多项式。在上面的代码中，`np.poly1d.divmod(q, p)` 同时返回了商和余数。如果你只想要商，可以像示例中那样只取 `divmod` 返回值的第一个元素。

1.4.2 多项式上的复杂运算

`np.poly1d` 对象提供了直接进行微分（求导）和积分运算的方法。这些方法分别是 `.deriv()` 和 `.integ()`。

微分（求导）使用 `.deriv(m=1, scale=1, rcond=None)` 方法可以对多项式进行微分。其中，`m` 指定了微分的阶数（默认为1），`scale` 是一个可选参数，用于在微分后调整多项式的系数（默认为1，即不调整），`rcond` 在这个上下文中不适用，因为 `np.poly1d` 的微分不涉及任何数值稳定性问题。

```
import numpy as np

# 创建一个多项式
p = np.poly1d([2, 3, 1]) # 表示 2x^2 + 3x + 1

# 对多项式进行一阶微分
dp = p.deriv()
print("一阶微分:", dp)

# 对多项式进行二阶微分
ddp = p.deriv(2)
print("二阶微分:", ddp)
```

积分使用 `.integ(m=1, k=0)` 方法可以对多项式进行积分。其中，`m` 指定了积分的阶数（默认为1），`k` 是积分常数，用于在每次积分后添加（默认为0）。

```
# 对多项式进行一阶积分
ip = p.integ()
print("一阶积分 (积分常数为0):", ip)

# 对多项式进行一阶积分，并指定积分常数为1
ip_k1 = p.integ(k=1)
print("一阶积分 (积分常数为1):", ip_k1)

# 注意：连续积分时，积分常数会累加
ipp = p.integ(2) # 这相当于两次应用.integ(k=0)
print("二阶积分 (积分常数为0):", ipp)

ipp_k2 = p.integ(2, k=2) # 这相当于两次应用.integ()，但总积分常数为2
print("二阶积分 (总积分常数为2):", ipp_k2)
```

请注意，在连续进行积分时，如果每次积分都指定了非零的积分常数，那么这些常数会累加。然而，在实际应用中，通常只在第一次积分时指定一个积分常数，而在后续的积分中保持该常数为零（除非有特定的理由需要改变它）。

此外，由于 `np.poly1d` 的积分和微分都是符号计算，因此它们不会产生数值误差（除非在后续的计算中使用了浮点数进行求值）。

`np.poly1d` 对象本身并没有直接提供求多项式根的方法，因为这是一个表示多项式的类，而不是进行符号数学计算的库。然而，你可以使用 NumPy 的 `numpy.roots` 函数来求取一个多项式（以系数形式给出）的根。

`numpy.roots` 函数接受一个包含多项式系数的数组（从最高次项到常数项），并返回多项式的根。这些根可能是实数或复数，具体取决于多项式。

下面是如何使用 `numpy.roots` 来求取由 `np.poly1d` 表示的多项式的根的示例：

```
import numpy as np

# 创建一个多项式
p = np.poly1d([2, 3, 1]) # 表示 2x^2 + 3x + 1

# 获取多项式的系数
coeffs = p.coef

# 使用 numpy.roots 求多项式的根
roots = np.roots(coeffs)

print("多项式的根:", roots)
```

在这个例子中，`p.coef` 返回了一个包含多项式系数的数组（在这个例子中是 `[2, 3, 1]`）。然后，我们将这个系数数组传递给 `numpy.roots` 函数，它返回了多项式的根。

需要注意的是，`numpy.roots` 函数返回的根是按照它们在复数平面上的位置来排序的，但这并不总是意味着它们按照模的大小或实部和虚部的某种顺序来排序。此外，如果多项式有重根，`numpy.roots` 也会返回相应的多个根（每个根都表示一次）。

还要注意的是，由于浮点数运算的精度限制，对于某些多项式（特别是高次多项式或具有复杂系数的多项式），`numpy.roots` 返回的根可能只是近似值，而不是精确值。因此，在需要高精度解的情况下，可能需要使用专门的符号数学库（如 SymPy）来求解多项式的根。

这里我可以补充一种如何从0开始计算一个方程数值解（不使用任何库函数）的方法。Newton法，也称为切线法，是一种寻找函数零点的有效方法。其思路是从一个初始估计值开始，不断迭代更新，直到找到零点或达到预定精度。

假设我们想要求解方程 $y = x^3 - x - 1$ 的零点，我们从一个初始点 x_0 开始。Newton法首先在 $x = x_0$ 处求函数的切线，并找到切线与x轴的交点 x_1 ，然后在 $x = x_1$ 处再次求切线，找到新的交点 x_2 ，如此不断迭代下去，最终会逼近 x_0 附近的零点。以下是用Python实现的 Newton法示例代码：

```

import numpy as np
def f(x):
    y=x**3-x-1#求根方程的表达式
    return y
def g(x):
    y=3*x**2-1#求根方程的导函数
    return y
def main():
    x_0=1.5 #取初值
    e=10**(-9) #误差要求
    L=0 #初始化迭代次数
    while L<100: #采用残差来判断
        x1=x_0-f(x_0)/g(x_0) #迭代公式,x(n+1)=x(n)-f(x(n))/f'(x(n))
        x_0=x1
        L=L+1 #统计迭代次数
        if abs(f(x_0)-0)<e:
            break
    print(f"x1={x1}") #输出数值解
    print(f(x_0)-0) # 验证解的正确性
    print(f"L={L}") #输出迭代次数
if __name__ == '__main__':
    main()

```

注意：Python中不需要像MATLAB那样使用特殊函数来控制结果的精度，因为Python的浮点数运算本身就具有较高的精度。

最终解得：

```
x1=1.3247179572447898 1.865174681370263e-13 L=4
```

程序能够利用 Newton法搜索到起始点最近的一个零点。

`numpy.polyfit` 函数用于拟合一个多项式 $p(x) = p[0] * x^{deg} + p[1] * x^{(deg-1)} + \dots + p[deg]$ 到给定的数据点，其中 `deg` 是多项式的度数。该函数返回多项式的系数，从最高次项到常数项。

下面是一个使用 `numpy.polyfit` 进行多项式拟合的详细示例：

```

import numpy as np
import matplotlib.pyplot as plt

# 示例数据: x值和对应的y值 (可能含有噪声)
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 0.8, 0.9, 0.1, -0.8, -1])

# 指定多项式的度数
deg = 2

# 使用numpy.polyfit进行多项式拟合
# 返回的coeffs是多项式的系数, 从最高次项到常数项
coeffs = np.polyfit(x, y, deg)

# 使用numpy.poly1d创建多项式对象 (可选)
p = np.poly1d(coeffs)

# 打印拟合得到的多项式
print("拟合得到的多项式:", p)

# 使用拟合得到的多项式进行预测 (可选)
x_fit = np.linspace(min(x), max(x), 100) # 生成一系列x值用于绘图
y_fit = p(x_fit) # 计算对应的y值

# 绘图展示原始数据和拟合曲线
plt.scatter(x, y, label='原始数据')
plt.plot(x_fit, y_fit, color='red', label='拟合曲线')
plt.xlabel('x')
plt.ylabel('y')
plt.title('多项式拟合示例')
plt.legend()
plt.show()

```

在这个例子中, 我们首先定义了x和y的示例数据 (这里y值是模拟的, 可能包含噪声)。然后, 我们指定了多项式的度数 `deg`, 并使用 `numpy.polyfit` 函数对x和y进行拟合, 得到了多项式的系数 `coeffs`。接着, 我们使用 `np.poly1d` (这是可选的) 创建了一个多项式对象 `p`, 它可以更方便地用于计算多项式的值或进行其他多项式运算。最后, 我们使用拟合得到的多项式对一系列x值进行预测, 并绘制了原始数据点和拟合曲线。

需要注意的是, 多项式拟合的度数 `deg` 是一个重要的参数, 它会影响拟合的效果。如果度数太低, 可能无法很好地拟合数据的复杂变化; 如果度数太高, 则可能会出现过拟合现象, 即模型在训练数据上表现

得很好，但在新的、未见过的数据上表现不佳。因此，在实际应用中，通常需要通过交叉验证等方法来选择合适的度数。